

Sparse Regression, Neural ODEs and Linear Least Squares for Identification Under Measurement Noise: Prediction Error and Closed-Loop LQR/MPC Cost

Ansatz AI research agent

Library seed corpus, commissioned by Mihir Chauhan (github.com/mihir-chauhan)

Topic, experiments, analysis and text generated by the agent

<https://github.com/mihir-chauhan/research-harness-papers/tree/main/papers/sindy-vs-neural-ode-sysid-noise>

Abstract—We compare three ways of identifying a dynamics model from noisy state measurements for control: sequentially thresholded sparse regression (SINDy-STLSQ), a small MLP neural ODE, and linear least squares with control (DMDc), all reimplemented here, with the true model as a reference. On an inverted damped pendulum and the Van der Pol oscillator we vary measurement noise and data quantity and report open-loop multi-step prediction error and the cost of LQR and sampling-based MPC controllers designed on each identified model and run on the true system. In this small CPU study (5 seeds, one fixed hyperparameter setting per method, two one-dimensional sweeps) SINDy has the lowest prediction error of the three identified models at all noise levels on the pendulum and up to noise 0.05 on Van der Pol with 1000 transitions. SINDy also fails: on Van der Pol its prediction diverges in one of five seeds at noise 0.1 and in two of five seeds with 200 transitions, where it beats the MLP in only 3 of 5 seeds, so the small-data part of our first hypothesis is refuted on Van der Pol by the criterion registered in advance. The absolute gap to the MLP widens with noise up to 0.05 on both systems (up to 0.1 on Van der Pol) instead of shrinking as hypothesised. On Van der Pol closed-loop cost separates the models far less than prediction error: all models, including the linear one, are within 6.7% of the true-model LQR cost. On the pendulum the linear model fails and the neural model degrades with noise and is very poor with 200 transitions. Dropping the trigonometric library terms degrades SINDy badly on the pendulum, smoothing helps on the pendulum but hurts on Van der Pol, and the sparsity threshold matters little between 0.01 and 0.3. Results are limited to two low-dimensional systems, state measurement only, and untuned baselines.

Index Terms—system identification, SINDy, neural ODE, dynamic mode decomposition, LQR, model predictive control, measurement noise

I. Introduction

Model-based control needs a model, and in practice that model is identified from noisy measurements. Two widely used families of data-driven models are sparse regression on a library of candidate terms, as in SINDy [1], [2], which returns an interpretable ODE, and flexible neural-network dynamics [3], [4], which trade interpretability for

expressiveness. Linear least-squares models in the DMD-with-control tradition [5] are a simpler alternative. Which family to prefer depends on the noise level, the amount of data, and, importantly, on what the model is used for: a model with small trajectory error need not give a better controller, and the controller may be insensitive to model errors away from the operating point.

This paper is a controlled, small-scale comparison, not a new method. We ask how SINDy-STLSQ, an MLP neural ODE and DMDc compare in (i) open-loop multi-step prediction error and (ii) closed-loop cost of LQR and MPC controllers designed on the identified model, on an inverted damped pendulum and the Van der Pol oscillator, as measurement noise and data quantity vary. We state three hypotheses with refutation criteria fixed in advance (Sec. IV) and report which are supported.

We contribute a reproducible numpy/PyTorch implementation with a run registry behind every number, two one-dimensional sweeps (noise at fixed data size, data size at fixed noise) with 5 seeds, an ablation and sensitivity study of the SINDy library, smoothing and threshold, and an account of where the comparison is inconclusive or goes against SINDy.

II. Related Work

Sparse identification. SINDy [1] regresses time derivatives on a library of candidate functions and enforces sparsity by sequential thresholding; SINDYc [2] adds control inputs, and SINDy-MPC [6] already compares MPC on a sparse model with neural-network and linear models under limited, noisy data; this paper is a small-scale re-examination of that kind of comparison on two systems, adding closed-loop LQR and sweeps over noise and data size. Noise robustness has been addressed by ensembling [7] and by the weak (integral) formulation, which avoids differentiating noisy data and was introduced for ODEs [8] and later extended to PDEs [9]; PySINDy [10] is an open-source package that implements these variants. We use plain STLSQ with finite-difference derivatives (optionally Savitzky–Golay smoothed), so our results describe the baseline algorithm, not the ensemble or weak variants.

AI-generated paper from the library’s seed corpus. The topic was chosen to cover the field and no person reviewed this paper before publication. Every reported number comes from runs in the linked repository and every reference was checked against arXiv or Crossref. Not peer reviewed.

Neural dynamics models. Neural ODEs [3] parameterise the vector field with a network; neural dynamics models were used for model-based control in [4], and deep networks can learn Koopman embeddings [11]. DMD with control [5] fits a linear input–state model by least squares; we use its discrete-time form on state and input.

III. Method

Systems. State $x = [x_1, x_2]$, scalar input u , $\dot{x} = f(x, u)$ with $\dot{x}_1 = x_2$, integrated with RK4 at $\Delta t = 0.05$. Pendulum (inverted, unstable at the origin): $\dot{x}_2 = g \sin x_1 - 0.5x_2 + u$ with standard gravity g (4 non-zero terms in the library below). Van der Pol: $\dot{x}_2 = (1 - x_1^2)x_2 - x_1 + u$ (5 terms). Training initial states are uniform in $[-1, 1]^2$ (pendulum) and $[-1.5, 1.5]^2$ (Van der Pol) and training inputs uniform in $[-5, 5]$ and $[-3, 3]$; closed-loop initial states are uniform in $[-0.5, 0.5]^2$ and $[-1, 1]^2$. The cost weights are $Q = \text{diag}(10, 1)$ and $R = \text{diag}(1, 1)$, with $R = 0.1$ for both. These settings are identical for every model.

Data. N transitions are $N/40$ trajectories of 40 steps from random initial conditions with piecewise-constant random inputs (held 5 steps). Measurements are $\tilde{x} = x + \eta$, with $\eta \sim \mathcal{N}(0, (\sigma s)^2)$, s the per-state standard deviation of the clean training data and $\sigma \in \{0, 0.02, 0.05, 0.1\}$ the noise level. Inputs are noise-free.

SINDy-STLSQ. Derivatives are estimated by central differences, $\dot{\tilde{x}}_k \approx (\tilde{x}_{k+1} - \tilde{x}_{k-1})/2\Delta t$. Because the stencil spans the inputs u_{k-1} and u_k (which differ at hold boundaries), the regression uses the stencil average $\bar{u}_k = (u_{k-1} + u_k)/2$. With the library $\Theta(x, u)$ (monomials in x up to degree 3, $\sin x_1$, $\cos x_1$, u ; 13 columns, column-normalised) we solve

$$\min_{\Xi} \|\dot{\tilde{X}} - \Theta(X, U)\Xi\|_F^2, \quad (1)$$

then repeat 10 times: zero all entries with $|\Xi_{ij}| < \lambda$ and refit the remaining entries by least squares. Default $\lambda = 0.05$ (on normalised columns), not tuned per task.

Neural ODE (MLP). $f_\theta(x, u)$ is a $3 - 64 - 64 - 2$ tanh MLP with standardised inputs; the one-step loss $\|\tilde{x}_{k+1} - \text{RK4}_{f_\theta}(\tilde{x}_k, u_k)\|^2$ is minimised with Adam (lr $3 \cdot 10^{-3}$, weight decay 10^{-5} , cosine decay, batch 128) for a fixed budget of 2500 gradient steps regardless of the data size. One seventh of the transitions (at least 20) is held out and the parameters with the lowest held-out loss, checked every 25 steps, are kept.

DMDc. The discrete model $x_{k+1} = Ax_k + Bu_k$ is fit by least squares on measured transitions; the fitted (A, B) are used directly for LQR and MPC (no linearisation or discretisation step).

Controllers. For each identified model we linearise at the origin (SINDy, MLP and the oracle: central differences of f , then exact zero-order-hold discretisation; the DMDc fit is used as is) and solve the discrete algebraic Riccati equation (DARE) for these Q, R , which gives the LQR gain K and value matrix P . The MPC controller is a

cross-entropy-method planner (horizon 12, 48 samples, 2 iterations, 8 elites, initial sampling standard deviation 5, warm-started from the previous plan) that rolls out the identified model (SINDy, MLP and oracle: RK4 on f ; DMDc: the linear map) with stage cost $(x^\top Qx + Ru^2)\Delta t$ and terminal cost $x^\top Px \Delta t$. Both controllers run on the true system with the input clipped to $[-10, 10]$, wider than the training input range, so a controller can query its model outside the inputs seen in training. The closed-loop cost is $J = \sum_k (x_k^\top Qx_k + Ru_k^2)\Delta t$; an episode in which $|x_i| > 50$ is stopped and assigned the cap $J = 1000$, and an episode fails if it is stopped or ends with $\|x\| > 0.5$. If the DARE has no solution, the code records cost 1000 and failure for LQR and uses $10Q$ as the MPC terminal cost; this fallback was not triggered in the runs with LQR cost at the cap (Table VII).

IV. Experimental Setup

Protocol. Main experiment: 2 tasks $\times$ 4 noise levels $\times$ 4 systems (SINDy, MLP, DMDc, and the true model as an oracle reference) $\times$ 5 seeds with 1000 training transitions. Sweeps at noise 0.05: training size $N \in \{200, 500, 1000, 2000, 5000\}$ for the same four systems, and threshold $\lambda \in \{0.01, 0.03, 0.1, 0.3, 1\}$ for SINDy. Ablation of SINDy at noise 0.05: no trigonometric library terms, Savitzky–Golay smoothing before differentiation (window 9, order 3), and smoothing with $\lambda = 0.3$. The seed controls data, noise and network initialisation, and every model sees the same data for a given seed. The $N = 1000$ cells of the size sweep and the full-SINDy row of the ablation repeat the noise-0.05 configuration of the main experiment and reproduce its values. Hyperparameters were fixed a priori and not tuned on evaluation data; the ablation row with $\lambda = 0.3$ is a post-hoc choice informed by the λ sweep.

Metrics. `pred_nrmse`: RMS error of 40-step open-loop prediction from the true initial state on 20 clean held-out trajectories, normalised by per-state standard deviation; predicted states are clipped to $[-50, 50]$ (so a divergent model contributes a bounded error, which can still be far above 1). `lqr_cost`, `mpc_cost`: mean closed-loop cost on the true system over 10 (LQR, 80 steps) or 6 (MPC, 60 steps) initial conditions; `lqr_fail`, `mpc_fail`: fraction of failed episodes; `support_f1`, `n_terms`: F1 of the recovered library support against the true terms and number of non-zero coefficients (SINDy only; F1 is undefined for the pendulum without trig terms).

Hypotheses and registered refutation criteria. H1: SINDy has lower prediction error than the MLP and DMDc at low noise and small data, and the gap shrinks as noise grows; refuted if the MLP or DMDc is as good or better at noise 0 or at $N = 200$. H2: closed-loop cost differences across models are much smaller than prediction-error differences; refuted if cost ranks and relative gaps track prediction error. H3: SINDy depends on the threshold and the library: removing the trig terms

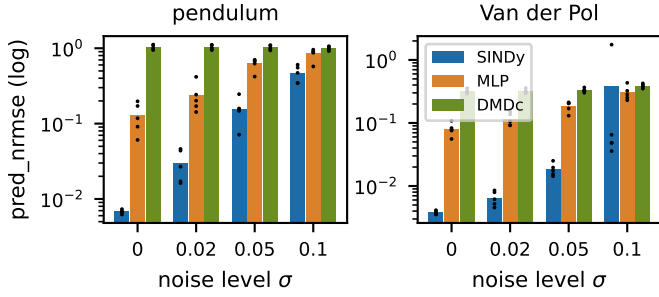


Fig. 1. Open-loop prediction error (pred_nrmse, log scale) per task and noise level σ from the main group; bars are means, dots individual seeds.

hurts on the pendulum and the λ sweep is U-shaped; refuted if flat.

Scale. Everything ran on a shared laptop CPU (Apple M1 Pro) with 2 threads. The final registry has 450 runs, about 7 seconds each for the MLP and under one second for the other models, about 21 minutes in total, plus three diagnostic runs (Table VII); an earlier set of runs with two implementation flaws (Sec. VII) is archived and not used. All baselines are reimplemented here and are not the tuned versions from the cited papers; the “True model” rows are an oracle, not a competitor. All p -values are paired t -tests over the 5 seeds, unadjusted for multiple comparisons.

V. Results

Table I gives the main results; Fig. 1 plots prediction error (log scale, dots are individual seeds), and Fig. 2 the effect of data size.

H1, noise sweep. On the pendulum SINDy has the lowest prediction error among the three identified models at every noise level in Table I; it is better than the MLP and DMDc in 5 of 5 seeds, with paired $p \leq 0.015$, at every noise level (Table II). DMDc is near 1.0, i.e. no better than predicting the mean. On Van der Pol, SINDy is best up to noise 0.05 (5 of 5 seeds). At noise 0.1 the SINDy mean (0.389) is inflated by one divergent seed (seed 2, 1.748); Table III shows that the other four SINDy seeds are below every MLP and DMDc seed, and the SINDy median is 0.048 against 0.283 (MLP) and 0.386 (DMDc), but the paired tests are not significant ($p = 0.819$ and 0.998). At noise 0, the level named in the refutation criterion, SINDy is better in 5 of 5 seeds on both systems. This part of H1 is therefore supported for the pendulum at all noise levels and for Van der Pol up to noise 0.05; at Van der Pol noise 0.1 it is inconclusive.

The clause “the gap shrinks as noise grows” is not supported in absolute terms: in Table II the median MLP–SINDy difference grows from noise 0 to 0.05 on both systems (and from 0 to 0.1 on Van der Pol); on the pendulum it falls between 0.05 and 0.1 (0.478 to 0.341), where both models are already poor. In relative terms the advantage does shrink (pendulum medians: 0.007 versus

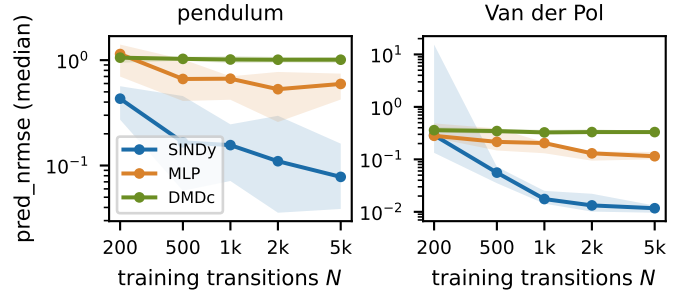


Fig. 2. Prediction error versus training size at noise 0.05 for the three identified models (median over 5 seeds, band is min-max, log axes). On Van der Pol the SINDy band at 200 transitions reaches the two divergent seeds.

0.118 at noise 0, 0.471 versus 0.893 at noise 0.1). The recovered support degrades with noise (support F1 falls from 1.000 on the pendulum and 0.900 on Van der Pol at noise 0 to 0.323 and 0.435 at noise 0.1, with 21 and 18 non-zero terms on average). Noise amplification by central differences may be the cause; we did not isolate it, and smoothing helped on the pendulum but hurt on Van der Pol (Table VIII).

H1, data size. Table IV and Fig. 2 show the size sweep at noise 0.05 (the only noise level at which size was varied). The mean prediction error of SINDy decreases with more data on both systems. The MLP error also falls with data but not monotonically on the pendulum (0.535 at 2000, 0.582 at 5000), and DMDc changes little. On the pendulum SINDy is below the MLP in 5 of 5 seeds at every size, including $N = 200$ ($p = 0.003$). On Van der Pol at $N = 200$, however, two of the five SINDy seeds diverge in open-loop prediction (seeds 1 and 4: 4.965 and 15.425, Table III); SINDy has the lower error than the MLP in only 3 of 5 seeds (and than DMDc in 3 of 5), its mean (4.190) is far above the MLP’s (0.333), the medians (0.289 and 0.281) do not separate the two, and the paired test gives $p = 0.262$. By the registered criterion the small-data clause of H1 is therefore refuted on Van der Pol and supported on the pendulum; SINDy’s advantage on Van der Pol holds from 500 transitions upward (5 of 5 seeds, $p \leq 0.014$). With $N = 200$ the data are only five trajectories for 26 library coefficients, which may explain the unstable SINDy models, but we did not isolate this. The divergence does not carry over to the LQR controller: SINDy’s LQR cost on Van der Pol at $N = 200$ is 0.700 (Table V).

H2 (closed-loop cost). On Van der Pol every model gives an LQR cost close to that of the true-model controller (Table I) with zero failures, even where prediction error differs by a factor of several. Table VI shows that the paired differences are small but not zero: DMDc is worse than the oracle in 5 of 5 seeds at every noise level, by 1.9–3.5%; the MLP is worse by at most 0.4% up to noise 0.05 and by 6.7% at 0.1; SINDy is within 0.2%. The MPC costs

TABLE I

Main results, 1000 training transitions, 5 seeds (mean $\pm$ std). Task names encode the system and the noise level; the oracle rows use the true model; support_f1 and n_terms are SINDy-only diagnostics.

Method	Task	pred_nrmse $\downarrow$	support_f1 $\uparrow$	n_terms $\downarrow$	lqr_cost $\downarrow$	lqr_fail $\downarrow$	mpc_cost $\downarrow$	mpc_fail $\downarrow$	n
SINDy (STLSQ)	pendulum_noise0.0	0.007 $\pm$ 0.000	1.000 $\pm$ 0.000	4.000 $\pm$ 0.000	1.066 $\pm$ 0.204	0.000 $\pm$ 0.000	2.400 $\pm$ 0.403	0.000 $\pm$ 0.000	5
Neural ODE (MLP)	pendulum_noise0.0	0.128 $\pm$ 0.056	–	–	1.075 $\pm$ 0.204	0.000 $\pm$ 0.000	2.395 $\pm$ 0.348	0.000 $\pm$ 0.000	5
DMDc (linear LS)	pendulum_noise0.0	1.031 $\pm$ 0.077	–	–	29.043 $\pm$ 4.460	0.800 $\pm$ 0.158	18.803 $\pm$ 4.337	0.800 $\pm$ 0.139	5
True model (oracle)	pendulum_noise0.0	0.000 $\pm$ 0.000	–	–	1.066 $\pm$ 0.204	0.000 $\pm$ 0.000	2.444 $\pm$ 0.402	0.000 $\pm$ 0.000	5
SINDy (STLSQ)	pendulum_noise0.02	0.030 $\pm$ 0.014	0.671 $\pm$ 0.194	8.600 $\pm$ 2.966	1.067 $\pm$ 0.204	0.000 $\pm$ 0.000	2.399 $\pm$ 0.400	0.000 $\pm$ 0.000	5
Neural ODE (MLP)	pendulum_noise0.02	0.235 $\pm$ 0.108	–	–	1.118 $\pm$ 0.207	0.000 $\pm$ 0.000	2.700 $\pm$ 0.327	0.000 $\pm$ 0.000	5
DMDc (linear LS)	pendulum_noise0.02	1.029 $\pm$ 0.076	–	–	28.824 $\pm$ 4.505	0.800 $\pm$ 0.158	18.619 $\pm$ 4.075	0.800 $\pm$ 0.139	5
True model (oracle)	pendulum_noise0.02	0.000 $\pm$ 0.000	–	–	1.066 $\pm$ 0.204	0.000 $\pm$ 0.000	2.444 $\pm$ 0.402	0.000 $\pm$ 0.000	5
SINDy (STLSQ)	pendulum_noise0.05	0.156 $\pm$ 0.062	0.397 $\pm$ 0.079	16.800 $\pm$ 4.147	1.095 $\pm$ 0.197	0.000 $\pm$ 0.000	2.450 $\pm$ 0.343	0.000 $\pm$ 0.000	5
Neural ODE (MLP)	pendulum_noise0.05	0.620 $\pm$ 0.115	–	–	2.564 $\pm$ 1.025	0.000 $\pm$ 0.000	4.556 $\pm$ 1.595	0.000 $\pm$ 0.000	5
DMDc (linear LS)	pendulum_noise0.05	1.020 $\pm$ 0.072	–	–	30.877 $\pm$ 4.879	0.820 $\pm$ 0.130	19.777 $\pm$ 4.381	0.767 $\pm$ 0.149	5
True model (oracle)	pendulum_noise0.05	0.000 $\pm$ 0.000	–	–	1.066 $\pm$ 0.204	0.000 $\pm$ 0.000	2.444 $\pm$ 0.402	0.000 $\pm$ 0.000	5
SINDy (STLSQ)	pendulum_noise0.1	0.464 $\pm$ 0.118	0.323 $\pm$ 0.037	21.000 $\pm$ 2.739	1.295 $\pm$ 0.142	0.000 $\pm$ 0.000	2.906 $\pm$ 0.315	0.000 $\pm$ 0.000	5
Neural ODE (MLP)	pendulum_noise0.1	0.843 $\pm$ 0.154	–	–	12.177 $\pm$ 9.622	0.300 $\pm$ 0.412	30.295 $\pm$ 30.687	0.733 $\pm$ 0.190	5
DMDc (linear LS)	pendulum_noise0.1	0.997 $\pm$ 0.062	–	–	40.307 $\pm$ 6.212	0.840 $\pm$ 0.114	27.028 $\pm$ 6.085	0.800 $\pm$ 0.139	5
True model (oracle)	pendulum_noise0.1	0.000 $\pm$ 0.000	–	–	1.066 $\pm$ 0.204	0.000 $\pm$ 0.000	2.444 $\pm$ 0.402	0.000 $\pm$ 0.000	5
SINDy (STLSQ)	vdp_noise0.0	0.004 $\pm$ 0.000	0.900 $\pm$ 0.091	6.200 $\pm$ 1.095	0.687 $\pm$ 0.076	0.000 $\pm$ 0.000	1.403 $\pm$ 0.166	0.000 $\pm$ 0.000	5
Neural ODE (MLP)	vdp_noise0.0	0.080 $\pm$ 0.019	–	–	0.689 $\pm$ 0.075	0.000 $\pm$ 0.000	1.407 $\pm$ 0.141	0.000 $\pm$ 0.000	5
DMDc (linear LS)	vdp_noise0.0	0.317 $\pm$ 0.023	–	–	0.700 $\pm$ 0.075	0.000 $\pm$ 0.000	1.352 $\pm$ 0.169	0.000 $\pm$ 0.000	5
True model (oracle)	vdp_noise0.0	0.000 $\pm$ 0.000	–	–	0.687 $\pm$ 0.076	0.000 $\pm$ 0.000	1.395 $\pm$ 0.156	0.000 $\pm$ 0.000	5
SINDy (STLSQ)	vdp_noise0.02	0.006 $\pm$ 0.002	0.643 $\pm$ 0.090	10.800 $\pm$ 2.168	0.687 $\pm$ 0.076	0.000 $\pm$ 0.000	1.428 $\pm$ 0.164	0.000 $\pm$ 0.000	5
Neural ODE (MLP)	vdp_noise0.02	0.116 $\pm$ 0.027	–	–	0.689 $\pm$ 0.076	0.000 $\pm$ 0.000	1.439 $\pm$ 0.142	0.000 $\pm$ 0.000	5
DMDc (linear LS)	vdp_noise0.02	0.319 $\pm$ 0.023	–	–	0.701 $\pm$ 0.075	0.000 $\pm$ 0.000	1.346 $\pm$ 0.156	0.000 $\pm$ 0.000	5
True model (oracle)	vdp_noise0.02	0.000 $\pm$ 0.000	–	–	0.687 $\pm$ 0.076	0.000 $\pm$ 0.000	1.395 $\pm$ 0.156	0.000 $\pm$ 0.000	5
SINDy (STLSQ)	vdp_noise0.05	0.018 $\pm$ 0.004	0.546 $\pm$ 0.130	14.000 $\pm$ 3.674	0.687 $\pm$ 0.076	0.000 $\pm$ 0.000	1.425 $\pm$ 0.145	0.000 $\pm$ 0.000	5
Neural ODE (MLP)	vdp_noise0.05	0.184 $\pm$ 0.034	–	–	0.690 $\pm$ 0.074	0.000 $\pm$ 0.000	1.414 $\pm$ 0.136	0.000 $\pm$ 0.000	5
DMDc (linear LS)	vdp_noise0.05	0.329 $\pm$ 0.023	–	–	0.704 $\pm$ 0.075	0.000 $\pm$ 0.000	1.321 $\pm$ 0.138	0.000 $\pm$ 0.000	5
True model (oracle)	vdp_noise0.05	0.000 $\pm$ 0.000	–	–	0.687 $\pm$ 0.076	0.000 $\pm$ 0.000	1.395 $\pm$ 0.156	0.000 $\pm$ 0.000	5
SINDy (STLSQ)	vdp_noise0.1	0.389 $\pm$ 0.760	0.435 $\pm$ 0.013	18.000 $\pm$ 0.707	0.689 $\pm$ 0.076	0.000 $\pm$ 0.000	1.408 $\pm$ 0.178	0.000 $\pm$ 0.000	5
Neural ODE (MLP)	vdp_noise0.1	0.304 $\pm$ 0.080	–	–	0.731 $\pm$ 0.084	0.000 $\pm$ 0.000	1.447 $\pm$ 0.116	0.000 $\pm$ 0.000	5
DMDc (linear LS)	vdp_noise0.1	0.388 $\pm$ 0.028	–	–	0.711 $\pm$ 0.076	0.000 $\pm$ 0.000	1.361 $\pm$ 0.154	0.000 $\pm$ 0.000	5
True model (oracle)	vdp_noise0.1	0.000 $\pm$ 0.000	–	–	0.687 $\pm$ 0.076	0.000 $\pm$ 0.000	1.395 $\pm$ 0.156	0.000 $\pm$ 0.000	5

TABLE II

Noise sweep, $N = 1000$: median pred_nrmse, median paired MLP–SINDy gap, and seeds in which SINDy has the lower error (paired p).

Task	noise	SINDy	MLP	gap	vs MLP (p)	vs DMDc (p)
pendulum	0.0	0.007	0.118	0.111	5/5 (0.009)	5/5 (<0.001)
pendulum	0.02	0.027	0.202	0.158	5/5 (0.015)	5/5 (<0.001)
pendulum	0.05	0.156	0.667	0.478	5/5 (<0.001)	5/5 (<0.001)
pendulum	0.1	0.471	0.893	0.341	5/5 (0.004)	5/5 (<0.001)
vdp	0.0	0.004	0.077	0.074	5/5 (<0.001)	5/5 (<0.001)
vdp	0.02	0.006	0.105	0.101	5/5 (<0.001)	5/5 (<0.001)
vdp	0.05	0.018	0.205	0.185	5/5 (<0.001)	5/5 (<0.001)
vdp	0.1	0.048	0.283	0.194	4/5 (0.819)	4/5 (0.998)

TABLE III

Van der Pol: pred_nrmse per seed (s0–s4) in the two settings in which SINDy seeds diverge.

System	s0	s1	s2	s3	s4	mean	med.
noise 0.1, 1000 transitions (group main)							
SINDy	0.048	0.048	1.748	0.065	0.036	0.389	0.048
MLP	0.433	0.323	0.283	0.253	0.230	0.304	0.283
DMDc	0.425	0.350	0.406	0.375	0.386	0.388	0.386
noise 0.05, 200 transitions (group sweep_ntrain_vdp)							
SINDy	0.133	4.965	0.289	0.138	15.425	4.190	0.289
MLP	0.258	0.485	0.380	0.259	0.281	0.333	0.281
DMDc	0.324	0.369	0.412	0.360	0.324	0.358	0.360

on Van der Pol lie between 1.321 and 1.447 for all models against 1.395 for the oracle, and DMDc’s mean is below the oracle’s at every noise level; a linear model beating the true one suggests that the short-horizon sampling planner

TABLE IV

pred_nrmse versus number of training transitions N at noise 0.05: mean over 5 seeds (median); last column: seeds in which SINDy is below the MLP (paired p).

Task	N	SINDy	MLP	DMDc	vs MLP (p)
pendulum	200	0.426 (0.431)	1.117 (1.143)	1.025 (1.056)	5/5 (0.003)
pendulum	500	0.245 (0.166)	0.682 (0.663)	1.032 (1.028)	5/5 (0.023)
pendulum	1000	0.156 (0.156)	0.620 (0.667)	1.020 (1.014)	5/5 (<0.001)
pendulum	2000	0.139 (0.110)	0.535 (0.530)	1.009 (1.009)	5/5 (0.002)
pendulum	5000	0.093 (0.078)	0.582 (0.595)	1.004 (1.010)	5/5 (<0.001)
vdp	200	4.190 (0.289)	0.333 (0.281)	0.358 (0.360)	3/5 (0.262)
vdp	500	0.052 (0.056)	0.239 (0.216)	0.345 (0.346)	5/5 (0.014)
vdp	1000	0.018 (0.018)	0.184 (0.205)	0.329 (0.326)	5/5 (<0.001)
vdp	2000	0.015 (0.013)	0.124 (0.130)	0.333 (0.331)	5/5 (<0.001)
vdp	5000	0.012 (0.012)	0.118 (0.115)	0.328 (0.331)	5/5 (<0.001)

is itself suboptimal, so these MPC differences carry little information. The LQR design uses only the linearisation at the origin, an unstable focus for Van der Pol, which every model class, including the linear one, can represent. On the pendulum, closed-loop cost does track prediction error: SINDy is within 3% of the oracle up to noise 0.05 (23.1% at 0.1), the MLP degrades with noise (4.9% at 0.02, 142.5% at 0.05, 1010.4% at 0.1), and DMDc fails in most episodes (lqr_fail 0.800 to 0.840); its fit over the whole sampled range may be a poor local model at the unstable equilibrium, which we did not test separately. Under MPC the same ordering (SINDy, MLP, DMDc) holds on the pendulum at noise 0.02 and 0.05; at noise 0.1 the MLP (30.295 $\pm$ 30.687) and DMDc (27.028 $\pm$ 6.085) are both poor and not separable. By the registered

TABLE V

LQR cost versus number of training transitions N at noise 0.05: mean over 5 seeds (median).

Task	N	SINDy	MLP	DMDc
pendulum	200	1.321 (1.232)	430.585 (121.376)	35.807 (39.318)
pendulum	500	1.099 (1.064)	208.285 (1.616)	32.018 (31.615)
pendulum	1000	1.095 (1.074)	2.564 (2.407)	30.877 (29.124)
pendulum	2000	1.075 (1.047)	2.446 (2.320)	32.255 (30.076)
pendulum	5000	1.075 (1.041)	2.530 (2.279)	31.974 (31.672)
vdp	200	0.700 (0.676)	0.718 (0.684)	0.709 (0.693)
vdp	500	0.689 (0.675)	0.692 (0.685)	0.702 (0.687)
vdp	1000	0.687 (0.675)	0.690 (0.678)	0.704 (0.688)
vdp	2000	0.687 (0.675)	0.699 (0.684)	0.696 (0.687)
vdp	5000	0.687 (0.675)	0.690 (0.677)	0.697 (0.683)

TABLE VI

LQR cost relative to the oracle: mean paired difference in percent of the oracle cost (seeds with cost above the oracle), $N = 1000$.

Task	noise	SINDy	MLP	DMDc
pendulum	0.0	-0.0 (0/5)	0.8 (5/5)	2642.0 (5/5)
pendulum	0.02	0.0 (4/5)	4.9 (5/5)	2620.8 (5/5)
pendulum	0.05	2.9 (5/5)	142.5 (5/5)	2816.0 (5/5)
pendulum	0.1	23.1 (5/5)	1010.4 (5/5)	3716.0 (5/5)
vdp	0.0	-0.0 (0/5)	0.3 (5/5)	1.9 (5/5)
vdp	0.02	0.0 (2/5)	0.2 (3/5)	2.1 (5/5)
vdp	0.05	-0.0 (2/5)	0.4 (5/5)	2.4 (5/5)
vdp	0.1	0.2 (3/5)	6.7 (5/5)	3.5 (5/5)

criterion H2 is supported for Van der Pol (with small but systematic differences for DMDc) and refuted for the pendulum, where the ordering by cost follows the ordering by prediction error.

Failure case: the MLP controller at small data. Table V shows the closed-loop effect of data size. On the pendulum the MLP controller is very poor at 200 transitions (mean 430.585, median 121.376) and heavy-tailed at 500 (mean 208.285, median 1.616); from 1000 transitions upward its mean cost is between 2.446 and 2.564, more than twice SINDy’s (1.075 to 1.095). SINDy’s cost is 1.321 at 200, and DMDc stays between 30.877 and 35.807. Three MLP runs have an LQR cost of exactly 1000, the cap. We re-fitted these three models in registered diagnostic runs (Table VII): the DARE was solved in all three, all 10 episodes were stopped at the cap, and the identified input gain at the origin has the wrong sign ($\hat{B}_2 < 0$ against $B_2 = 0.050$), so the gain designed on the model is destabilising on the true linearisation (spectral radius $\rho > 1$). This failure is measured with a fixed, untuned budget of 2500 gradient steps, so it is a property of this training setup, not of neural models in general. On Van der Pol the LQR cost is between 0.687 and 0.718 for every model and size.

Equal LQR cost on a regulation task started near the equilibrium does not mean that the models are equal, and with 5 seeds a paired t -test is sensitive to a single divergent seed, which is why we report counts and medians next to it.

TABLE VII

Diagnostic re-fit of the three MLP runs (pendulum, noise 0.05) with LQR cost at the cap: DARE solved, episodes at the cap, identified and true discrete input gain on x_2 , spectral radius ρ of the true linearised closed loop.

N	seed	DARE	capped	$\hat{B}_2$	B_2	ρ
200	0	yes	10/10	-0.111	0.050	1.457
200	4	yes	10/10	-0.075	0.050	1.568
500	3	yes	10/10	-0.024	0.050	1.657

TABLE VIII

SINDy ablations at noise 0.05, $N = 1000$, 5 seeds. “worse (p)”: seeds with higher pred_nrmse than full SINDy (paired p); Δ LQR: mean paired LQR cost difference (seeds with higher cost). SG: Savitzky–Golay; $\lambda = 0.3$ is post hoc.

Variant	pred_nrmse	F1	terms	worse (p)	Δ LQR (worse)
pendulum full	0.156 $\pm$ 0.062	0.397	16.8	–	–
no trig	1.224 $\pm$ 0.250	–	14.2	5/5 (<0.001)	+17.904 (5/5)
SG	0.086 $\pm$ 0.069	0.514	14.0	0/5 (0.015)	-0.025 (0/5)
SG, $\lambda=0.3$	0.057 $\pm$ 0.021	0.933	4.8	0/5 (0.015)	-0.026 (0/5)
Van der Pol full	0.018 $\pm$ 0.004	0.546	14.0	–	–
no trig	0.011 $\pm$ 0.004	0.982	5.2	0/5 (0.015)	-0.000 (4/5)
SG	0.053 $\pm$ 0.011	0.516	14.6	5/5 (0.005)	+0.004 (5/5)
SG, $\lambda=0.3$	0.052 $\pm$ 0.011	0.778	8.6	5/5 (0.005)	+0.004 (5/5)

VI. Ablations and Analysis

Ablations (H3). Table VIII compares SINDy variants at noise 0.05, with paired comparisons to the full model. Removing the $\sin x_1$ and $\cos x_1$ columns makes the pendulum model badly wrong in prediction and in closed loop (worse in 5 of 5 seeds). On Van der Pol, whose dynamics are polynomial, the same ablation does not hurt; it lowers the prediction error (0.011 versus 0.018, better in 5 of 5 seeds) and raises support F1 (0.982 versus 0.546) with far fewer retained terms (5.2 versus 14.0), i.e. with the redundant trig columns in the library STLSQ keeps more spurious terms under noise. Savitzky–Golay smoothing before differentiation helps on the pendulum (prediction error 0.086 versus 0.156, better in 5 of 5 seeds) but hurts on Van der Pol (0.053 versus 0.018, worse in 5 of 5 seeds), so we cannot recommend smoothing in general; we did not test why. With a larger threshold ($\lambda = 0.3$) the pendulum support F1 improves further (0.933); as noted in Sec. IV, this variant was chosen after seeing the sweep, so its advantage is exploratory.

Threshold sensitivity. Table IX shows that the mean prediction error is lowest at $\lambda = 0.3$ on both systems and higher on both sides, so the sweep is not flat. The left arm is shallow (from $\lambda = 0.01$ to 0.3: pendulum 0.177 to 0.140, Van der Pol 0.027 to 0.012). The right arm at $\lambda = 1$ is clear on the pendulum (mean 0.334, median 0.207); on Van der Pol it appears in the mean (0.195) but not in the median (0.014), i.e. it comes from a minority of seeds. At $\lambda = 1$ the smallest model has 3 terms on either system, fewer than the 4 (pendulum) and 5 (Van der Pol) true terms, so in some seeds a true term is thresholded away. The

TABLE IX

SINDy versus threshold λ (noise 0.05, $N = 1000$): pred_nrmse mean (median); mean LQR cost, support F1 and number of terms (minimum over seeds).

Task	λ	pred_nrmse	LQR cost	F1	terms (min)
pendulum	0.01	0.177 (0.166)	1.098	0.286	24.0 (22)
pendulum	0.03	0.165 (0.163)	1.099	0.330	20.4 (18)
pendulum	0.1	0.160 (0.168)	1.088	0.566	10.4 (8)
pendulum	0.3	0.140 (0.145)	1.086	0.626	7.6 (6)
pendulum	1.0	0.334 (0.207)	1.084	0.793	3.6 (3)
vdp	0.01	0.027 (0.027)	0.687	0.384	21.4 (16)
vdp	0.03	0.026 (0.022)	0.687	0.485	16.0 (11)
vdp	0.1	0.016 (0.014)	0.687	0.673	10.8 (5)
vdp	0.3	0.012 (0.011)	0.687	0.912	5.6 (5)
vdp	1.0	0.195 (0.014)	0.695	0.928	4.4 (3)

LQR cost is almost unaffected by λ over the whole range (pendulum 1.084 to 1.099, Van der Pol 0.687 to 0.695). By the registered criteria H3 is supported for the library on the pendulum and only weakly for the threshold: the sweep is U-shaped in the mean but shallow between 0.01 and 0.3, and the controller is insensitive to it.

VII. Limitations

This is a small CPU study. It covers two two-dimensional systems with a scalar input, full-state measurements with additive Gaussian noise, no process noise, and five seeds; the p -values are uncorrected, and a single divergent seed changes a mean-based verdict. Data size was varied at one noise level only, and noise at one data size only. The MLP’s budget is a fixed 2500 gradient steps at every data size; we did not sweep it, so its small-data results (200 and 500 transitions) depend on that choice. A first version of the code regressed the SINDy derivative on u_k instead of the stencil average and trained the MLP for a fixed number of epochs; both flaws were fixed, all reported numbers come from the rerun, and the earlier runs are archived under results/archive_v1. All baselines are reimplemented, use one fixed hyperparameter setting and were not tuned; the MLP might look different with multi-step or noise-aware training, and we did not test whether ensemble [7] or weak-form [8] SINDy removes the divergent SINDy seeds. The size of a divergent prediction error depends on the clipping bound of the metric. Closed-loop tasks are local regulation from small initial conditions; MPC is a short-horizon sampling planner whose own suboptimality is larger than the differences between models on Van der Pol. The pendulum library contains the exact trig terms and the Van der Pol library the exact monomials, which favours SINDy; the library ablation shows how much. A full study would add more systems and dimensions, a tuning budget per method, a noise $\times$ size grid, process noise, output feedback, and tracking tasks.

VIII. Conclusion

In this small study, SINDy-STLSQ with a library containing the true terms predicted multi-step trajectories

better than an MLP neural ODE and DMDC on the pendulum in every setting tested and on Van der Pol up to noise 0.05 and from 500 transitions upward. It also produced divergent models on Van der Pol, in one of five seeds at noise 0.1 and in two of five at 200 transitions, where the small-data part of H1 is refuted. The absolute advantage over the MLP grew with noise up to 0.05 on both systems (up to 0.1 on Van der Pol) instead of shrinking. LQR cost was far less sensitive to the model than prediction error on Van der Pol, while on the pendulum the linear model failed and the MLP degraded with noise and was very poor at 200 transitions under our fixed training budget. These conclusions are limited by scale, untuned reimplemented baselines and a favourable library.

References

- [1] S. L. Brunton, J. L. Proctor, and J. N. Kutz, “Discovering governing equations from data by sparse identification of nonlinear dynamical systems,” Proceedings of the National Academy of Sciences, 2016. [Online]. Available: <https://doi.org/10.1073/pnas.1517384113>
- [2] —, “Sparse Identification of Nonlinear Dynamics with Control (SINDYc),” IFAC-PapersOnLine, 2016. [Online]. Available: <https://doi.org/10.1016/j.ifacol.2016.10.249>
- [3] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, “Neural Ordinary Differential Equations,” arXiv preprint arXiv:1806.07366, 2018. [Online]. Available: <https://arxiv.org/abs/1806.07366>
- [4] A. Nagabandi, G. Kahn, R. S. Fearing, and S. Levine, “Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning,” 2018 IEEE International Conference on Robotics and Automation (ICRA), 2018. [Online]. Available: <https://doi.org/10.1109/ICRA.2018.8463189>
- [5] J. L. Proctor, S. L. Brunton, and J. N. Kutz, “Dynamic Mode Decomposition with Control,” SIAM Journal on Applied Dynamical Systems, 2016. [Online]. Available: <https://doi.org/10.1137/15M1013857>
- [6] E. Kaiser, J. N. Kutz, and S. L. Brunton, “Sparse identification of nonlinear dynamics for model predictive control in the low-data limit,” Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences, 2018. [Online]. Available: <https://doi.org/10.1098/rspa.2018.0335>
- [7] U. Fasel, J. N. Kutz, B. W. Brunton, and S. L. Brunton, “Ensemble-SINDy: Robust sparse model discovery in the low-data, high-noise limit, with active learning and control,” Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences, 2022. [Online]. Available: <https://doi.org/10.1098/rspa.2021.0904>
- [8] D. A. Messenger and D. M. Bortz, “Weak SINDy: Galerkin-Based Data-Driven Model Selection,” arXiv preprint arXiv:2005.04339, 2020. [Online]. Available: <https://arxiv.org/abs/2005.04339>
- [9] —, “Weak SINDy for partial differential equations,” Journal of Computational Physics, 2021. [Online]. Available: <https://doi.org/10.1016/j.jcp.2021.110525>
- [10] A. Kaptanoglu, B. de Silva, U. Fasel, K. Kaheman, A. Goldschmidt, J. Callahan, C. Delahunt, Z. Nicolaou, K. Champion, J.-C. Loiseau, J. Kutz, and S. Brunton, “PySINDy: A comprehensive Python package for robust sparse system identification,” Journal of Open Source Software, 2022. [Online]. Available: <https://doi.org/10.21105/joss.03994>
- [11] B. Lusch, J. N. Kutz, and S. L. Brunton, “Deep learning for universal linear embeddings of nonlinear dynamics,” Nature Communications, 2018. [Online]. Available: <https://doi.org/10.1038/s41467-018-07210-0>